blocks/clean_room.rb
```
class CleanRoom
  def current_temperature
    # ...
  end
end

clean_room = CleanRoom.new
clean_room.instance_eval do
  if current_temperature < 20
    # TODO: wear jacket
  end
end
```

A Clean Room is just an environment where you can evaluate your blocks. It can expose a few useful methods that the block can call, such as current_temperature in the example above. However, the ideal Clean Room doesn't have many methods or instance variables, because the names of those methods and instance variables could clash with the names in the environment that the block comes from. For this reason, instances of BasicObject usually make for good Clean Rooms, because they're *Blank Slates (66)*—so they barely have any method at all.

(Interestingly, BasicObject is even cleaner than that: in a BasicObject, standard Ruby constants such as String are out of scope. If you want to reference a constant from a BasicObject, you have to use its absolute path, such as ::String.)

You'll find a practical example of a Clean Room in *Quiz: A Better DSL*, on page 98.

That's all you have to know about instance_eval. Now you can move on to the last topic in today's roadmap: callable objects.

Callable Objects

Where you learn how blocks are just part of a larger family, and Bill shows you how to set code aside and execute it later.

If you get to the bottom of it, using a block is a two-step process. First, you set some code aside, and second, you call the block (with yield) to execute the code. This "package code first, call it later" mechanism is not exclusive to blocks. There are at least three other places in Ruby where you can package code:

- In a *proc*, which is basically a block turned object
- In a *lambda*, which is a slight variation on a proc
- In a *method*